

Day 5

Trust, evaluation & Demo Day

"It works" is a claim, not a fact.

The week, so far

Day	You built	Standing on
1	search + extraction, by hand	tokens, vectors, schemas
2	an agent calling your tools	the model-tool loop
3	memory, RAG, guardrails	state, callbacks
4	a report-writing team	workflow agents
5	the reason to believe any of it	evals + traces

Today ends with **your demo**. Everything before lunch is about making that demo *credible*.

Evaluation · Tracing · Discipline + capstone · Demo Day 🎬

Part 1

"It works" is a **claim**

Why agents are hard to trust

Run the same question 3× → 3 different tool sequences, 3 wordings, (hopefully) one conclusion.

- **Nondeterminism:** temperature, sampling: Day 1, slide one, coming home
- **Compounding:** 6 agents × 95% reliable each ≈ 74% end-to-end
- **Silent failure:** a wrong-but-fluent report *looks identical* to a right one

Classic tests assert `output == expected`. Dead on arrival here.

So what do we assert? **Behavior.**

An eval case = a behavior, written down

```
{  
  "user_content": "Did the devs of Silent Depths fix the save bug?",  
  "tool_uses":    [search_reviews(...), search_docs(...)], ← expected path  
  "final_response": "Yes: patch v1.2, January 2026 ..." ← reference  
}
```

Today's metrics (local, no judge model):

Metric	Asserts	Method
tool_trajectory_avg_score	right tools, right args	exact match
response_match_score	close to reference wording	ROUGE overlap

```
adk eval playfield_agent evals/analyst_smoke.evalset.json \  
  --config_file_path evals/test_config.json --print_detailed_results
```

You will see failures. Good.

A failing eval means one of **three** things:

1. the **agent** is wrong → fix instruction/tools
2. the **expectation** is wrong → fix the case
3. the **metric** is wrong → exact-match trajectory vs a fuzzy search query is merciless; that's what LLM-judged metrics (`final_response_match_v2`, rubrics) are for

Diagnosing *which one* is the skill.

A green suite you don't understand is worse than a red one you do.

Walkthrough: Part 1

-  same question 3×: what varies, what doesn't
-  read the 4 smoke cases (incl. the **guardrail** case: zero tools expected)
-  `adk eval`, then autopsy every failure: agent, expectation, or metric?



Break

10 minutes

Part 2

Tracing: what did it actually do?

From "it seems confused" to a line number

The trace shows, per step: the exact request (instruction + history + tool declarations), the response, every tool call with args, tokens, latency.

Debugging discipline: find the **first wrong step**, fix *that layer*:

First wrong step	The layer to fix
wrong tool chosen	tool docstring, or instruction routing
right tool, bad args	docstring examples
right data, wrong conclusion	instruction (synthesis rules)
never looked again after a miss	instruction ("rephrase and retry")

Then **re-run the eval** and watch the score move. eval → trace → fix → eval.

Walkthrough: Part 2

-  trace the save-bug question end to end
-  autopsy one real miss; fix the layer, not the symptom
-  pin it: add an eval case for the behavior you fixed
-  remember `log_tool_calls` from Day 3? That was tracing, hand-rolled



Lunch

30 minutes · after: the checklist, then you build

Part 3

The discipline, then the **capstone**

The trust checklist (take-home)

1. **Grounding**: facts from tools, never model memory (*D1, D3*)
2. **Structure**: schemas at every model boundary (*D1, D4*)
3. **Honest failure**: error dicts; "not found" is an answer (*D2, D3*)
4. **Memory by design**: explicit, prefixed, inspectable state (*D3*)
5. **Guardrails in layers**: callback → tool → instruction (*D3*)
6. **Small jobs**: one agent, one competence (*D4*)
7. **Bounded loops**: every cycle has an exit and a cap (*D4*)
8. **Evals + traces**, or you're shipping vibes (*D5*)

Rule 9: know when NOT to add an agent

An agent earns its place only when a **decision must be made in language**.

You need	You want	Not
fixed transformation	a function	an agent
fixed order	a pipeline	an agent
extract fields	a schema call	an agent
route by judgment	ok, an agent	a big one

Your Day-4 system is mostly *not* agents: deterministic skeleton, judgment only at the joints. That's why it works.

Every agent you don't add is one you don't have to trust.

Capstone (45 min, then demos)

day5/CAPSTONE.md : pick an extension, or pitch your own:

comparison reports · trend watch · the skeptic critic · Romanian mode ·
eval hardening · live-web researcher

The bar, non-negotiable:

1. **runs live** on a question the room could ask
2. **evaluated**: ≥ 3 cases; failures fine *if you can explain them*
3. **traceable**: one trace you can narrate

Priorities: eval > feature polish. One great run > three shaky features.



Capstone work time. Mentors are circulating.

Part 4

 **Demo Day**

The format: 3 minutes, live

1. **The question** your system answers · 10 sec
2. **The run**: one real query, narrated as it executes · 90 sec
3. **The proof**: eval scorecard and/or one trace · 45 sec
4. **The scar**: best bug of the week, what it taught you · 15 sec

Then one question from the room.

While watching: note **one thing you'd steal**. We'll say them out loud; it's the best compliment an engineer can give.

Day 1 → Day 5

On Day 1 you asked a chatbot what an AI agent is.

This afternoon you're demoing: retrieval over two corpora, structured extraction, durable memory, guardrails, a parallel research team, a self-critiquing writer, **with an eval suite and traces to prove it.**

The dataset was fictional. The skills are not.

Thank you 🙌

Now go build something real.

(and put the capstone on GitHub; it's a portfolio piece)

Alex · Radu · Radu. We'll stick around for questions.